RAMNIRANJAN JHUNJHUNWALA COLLEGE
(DEPARTMENT OF DSAI)
Ghatkopar (West), Mumbai - 86

Project Report
On

Workout Counter with Mediapipe

In partial fulfillment of M.Sc. (DSAI)

By
Mr. Pawan Kumar Yadav

M.Sc. DSAI
University of Mumbai
(2021-2022)

RAMNIRANJAN JHUNJHUNWALA COLLEGE (AUTONOMOUS)

(Affiliated to University of Mumbai)

GHATKOPAR(WEST), 400086.

Certificate

NAAC Re-Accredited "A" Grade
विद्यया अमृतं अश्नुते
ESTD. 1963
RJC
ISO 9001:2008
Best College
UNIVERSITY of MUMBAI

This is to certify that the Project entitled, "Workout Counter with Mediapipe" is bonafide work of Mr. Pawan Kumar Yadav bearing Seat No: - 19 submitted in partial fulfillment of the requirements for the award of Degree Master of Science in Data Science and Artificial Intelligence.

Signature of Internal Guide

Sign of Co-ordinator

Examiner

16 JUL 2022

Date: *College Seal*

TABLE OF CONTENTS

Workout Counter with Mediapipe

1. ACKNOWLEDGEMENT

1. ACKNOWLEDGEMENT

Before we get into the thick of things, we would like to add a few heartfelt words for the people who were part of the **Workout Counter with Mediapipe** project in numerous ways, people who gave unending support right from the stage the project idea was conceived.
The four things that go on to make a successful endeavor are dedication, hard work, patience and correct guidance. Able and timely guidance not only helps in making an effort fruitful but also transforms the whole process of learning and implementing into an enjoyable experience.
In particular, I would like to thank our college director **Dr. Usha Mukundan**.

I would like to give a very special honor and respect to our data science teachers who took keen interest in checking the minute details of the project work and guided us throughout the same. A sincere quote of thanks to the non-teaching staff for providing us software & their time.

2. ABSTRACT

2. ABSTRACT

If we search online for workout applications, we get many results with multiple functionalities. These applications provide many workout programs which help us to perform it on our own. But somewhere those programs cannot improve the user's posture and accuracy. Those applications aim to provide workouts only, but these do not track any activity of the user which can help a user to count and track their workout. To avoid such a problem, I created a web app which counts workout using the pose classification technique. The objective is to develop an application that can count workout performing various exercises without any mistakes. An application with the pre-trained workout set with the help of pose estimation technique. We proposed to use the Blaze-pose pose estimation module developed by MediaPipe. This neural network provides 33 body-points which are more than enough to capture the movements of the user. The pose estimation model is generally used to classify the different movements. We are using such technology with some advancement to provide accuracy of the user's workout which will provide state of art results.

3. INTRODUCTION

3. INTRODUCTION

Nowadays, it is observed that a healthy lifestyle is one of the most crucial things in day-to-day life. Everyone wants to be fit and healthy but it is also seen that very few of them follow the routine strictly. From our observation we notice that many of the people are willing to do exercises but sometimes they don't have enough knowledge about the workouts and their forms, variations, etc. There are hundreds of different workout videos on the internet that include short workout videos for a variety of exercises. The purpose of these programs is to assist users in performing those exercises on their own. Despite having these features, they lack the ability to monitor user's workout.

New data are emerging that exercise may reduce the risk of acute respiratory distress syndrome, a major cause of death in patients with Coronavirus disease 2020 (COVID-19). Just as COVID-19 changed the way health care is, it has also upended the way consumers approach physical activity. Although gyms and workout classes are crowded areas with lots of surface areas, they can transmit infections. To avoid these issues, gyms and fitness centers throughout the country were closed during this period to ensure safety.

To overcome such a problem, we are trying to create an application to encourage users to take a more active interest in their health. For this application, we propose to use the blaze pose machine learning model provided by mediapipe, which can run the application on very few processing-powered devices. Due to the rapid development of deep convolutional neural networks, human pose estimation has significant performance improvement, this helps to accurately analyze the exercise the user is performing.

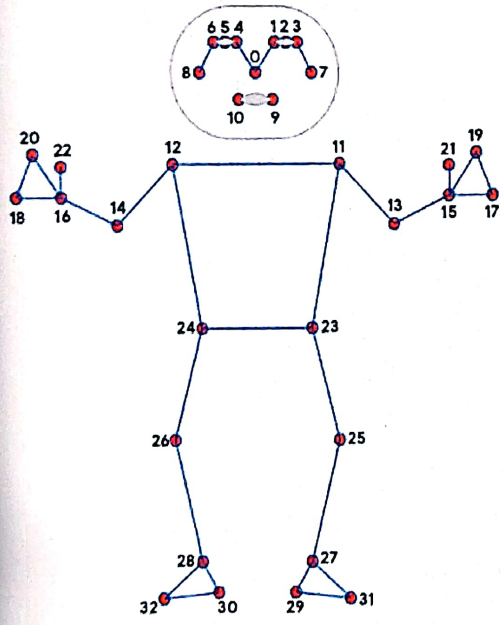
Human pose estimation localizes body key points to accurately recognize the postures of individuals given in images or video. This step is a crucial prerequisite for the workout analysis. We describe a method of detecting a user's body posture during a workout, and comparing their body posture to a professional reference workout, for aiding in resolving this problem. Based on the latest advancement in deep learning for human body pose estimation, we represent the human body as a collection of limbs and calculate angles between them to detect errors and provide accuracy to the user.

4. LITERATURE SURVEY

4. LITERATURE SURVEY

Human pose estimation refers to the process of inferring poses in an image/ video and these estimations are performed in 2D and 3D. We identified various techniques from past years which are used for human pose recognition. Many of the researchers proposed that the new system collaborated with past technologies which give the state of art results. Nowadays, deep convolutional neural networks provide dominant solutions. There are two mainstream methods: Regression the position of key-points and estimating key-point heat maps followed by choosing the locations with the highest heat values as the key-points.

The paper presented by Ching-Hang Chen in the year 2017 used 2D pose estimation, followed by 3D exemplar matching it uses Big Data sets of 3D mocap data to regress 3D pose from 2D measurements.



0. nose
1. left_eye_inner
2. left_eye
3. left_eye_outer
4. right_eye_inner
5. right_eye
6. right_eye_outer
7. left_ear
8. right_ear
9. mouth_left
10. mouth_right
11. left_shoulder
12. right_shoulder
13. left_elbow
14. right_elbow
15. left_wrist
16. right_wrist
17. left_pinky
18. right_pinky
19. left_index
20. right_index
21. left_thumb
22. right_thumb
23. left_hip
24. right_hip
25. left_knee
26. right_knee
27. left_ankle
28. right_ankle
29. left_heel
30. right_heel
31. left_foot_index
32. right_foot_index

Pic Credit : https://google.github.io/mediapipe/images/mobile/pose_tracking_full_body_landmarks.png

Juliet Martinez et al in the year 2017 had used a 2d-to-3d human pose estimation and coupled with a state-of-the-art 2d detector Qingtian Yu, Haopeng Wang, Fedwa Lamartine, and Abdulmotaleb El Saddik have proposed a system comprising a deep-learning multitask model of exercise recognition and repetition counting. Multitask system can estimate human pose, identify physical activities and count repeated motions with 95.69% accuracy in exercise recognition on the Rep-Penn dataset. The multitask model also performed well in repetitive counting with 0.004 Mean Average Error (MAE) and 0.997 Off-By-One (OBO) accuracy on the Rep-Penn dataset. A paper by Choi in the year 2021 proposed a mobile-friendly model, Mobile Human Pose, for real-time 3D human pose estimation from a single RGB image. It consists of the modified MobileNetV2 backbone, a parametric activation function, and the skip concatenation inspired by U-Net. This model is seven times smaller than the ResNet-50 based model. In addition, their extra small model reduces inference time by 12.2ms on Galaxy S20 CPU, which is suitable for real-time 3D human pose estimation in mobile applications.

Xie et al in the year of 2019 have proposed a human pose estimation algorithm called Open Pose But its efficiency is very low. To overcome this, deep learning methods are based on Tensor Flow to recognize human body postures.

5. HUMAN POSE ESTIMATE

5. Human Pose Estimation

Human Pose Estimation (HPE) is a way of identifying and classifying the joints in the human body.

Essentially it is a way to capture a set of coordinates for each joint (arm, head, torso, etc.,) which is known as a key point that can describe a pose of a person. The connection between these points is known as a pair.

The connection formed between the points has to be significant, which means not all points can form a pair. From the outset, the aim of HPE is to form a skeleton-like representation of a human body and then process it further for task-specific applications.

However,

There are three types of approaches to model the human body:

- Skeleton-based model
- Contour-based model
- Volume-based model

Pose estimation is a machine learning task that estimates the pose of a person from an image or a video by estimating the spatial locations of specific body parts (key points). Pose estimation is a computer vision technique to track the movements of a person or an object. This is usually performed by finding the location of key points for the given objects. Based on these key points we can compare various movements and postures and draw insights.

5.1. MediaPipe

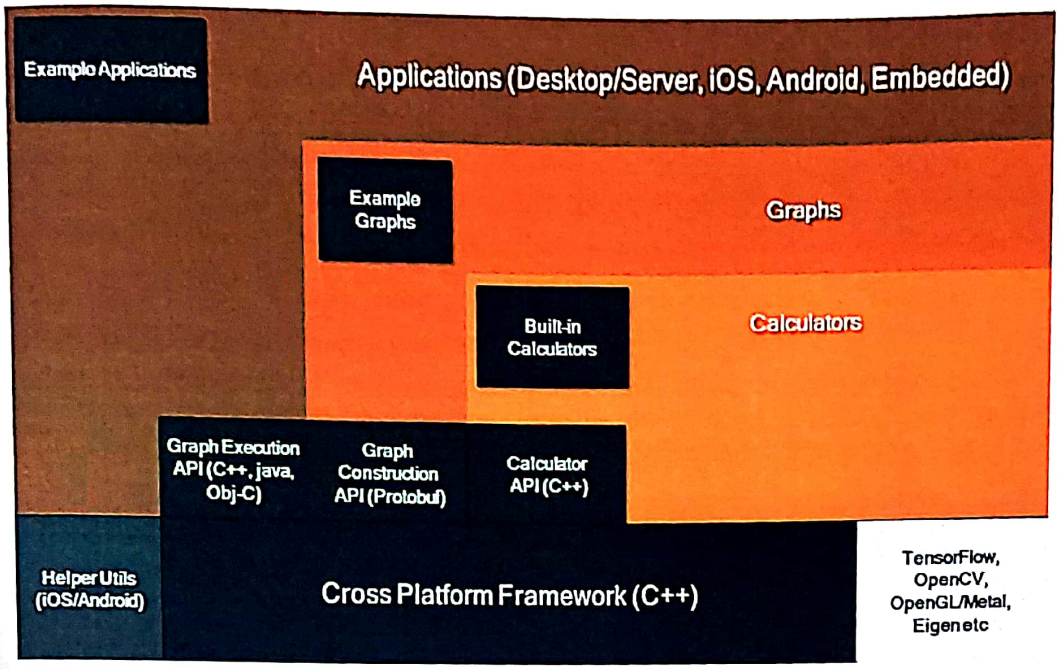
MediaPipe powers revolutionary products and services we use daily. Unlike power-hungry machine learning Frameworks, MediaPipe requires minimal resources. It is so tiny and efficient that even embedded IoT devices can run it. In 2019, MediaPipe opened up a whole new world of opportunity for researchers and developers following its public release.

5.1.1. What is MediaPipe?

MediaPipe is a Framework for building machine learning pipelines for processing time-series data like video, audio, etc. This cross-platform Framework works in Desktop/Server, Android, iOS, and embedded devices like Raspberry Pi and Jetson Nano

5.1.2. MediaPipe Toolkit

MediaPipe Toolkit comprises the Framework and the Solutions. The following diagram shows the components of the MediaPipe Toolkit.



2.1 Framework

The Framework is written in C++, Java, and Obj-C, which consists of the following APIs.

Calculator API (C++).

Graph construction API (Protobuf).

Graph Execution API (C++, Java, Obj-C).

Graphs

The MediaPipe perception pipeline is called a Graph.

In computer science jargon, a graph consists of Nodes connected by Edges. Inside the MediaPipe Graph, the nodes are called Calculators, and edges are called Streams. Every stream carries a sequence of Packets that have ascending time stamps.

In the image above, we have represented Calculators with rectangular blocks and Streams using arrows.

Calculators

These are specific computation units written in C++ with assigned tasks to process. The packets of data (Video frame or audio segment) enter and leave through the ports in a calculator. When initializing a calculator, it declares the packet payload type that will traverse the port. Every time a graph runs, the Framework implements Open, Process, and Close methods in the calculators. Open initiates the calculator; the process repeatedly runs when a packet enters. The process is closed after an entire graph run.

As an example, consider the first calculator shown in the above graph. The calculator, ImageTransform, takes an image at the input port and returns a transformed image in the output port. On the other hand, the second calculator, ImageToTensor, takes an image as input and outputs a tensor.

Calculator Types in MediaPipe

All the calculators shown above are built-in into MediaPipe. We can group them into four categories.

- Pre-processing calculators are a family of image and media processing calculators. The ImageTransform and ImageToTensors in the graph above fall in this category.
- Inference calculators allow native integration with Tensorflow and Tensorflow Lite for ML inference.
- Post-processing calculators perform ML post-processing tasks such as detection, segmentation, and classification. TensorToLandmark is a post-processing calculator.
- Utility calculators are a family of calculators performing final tasks such as image annotation.
- The calculator APIs allow you to write your custom calculator.

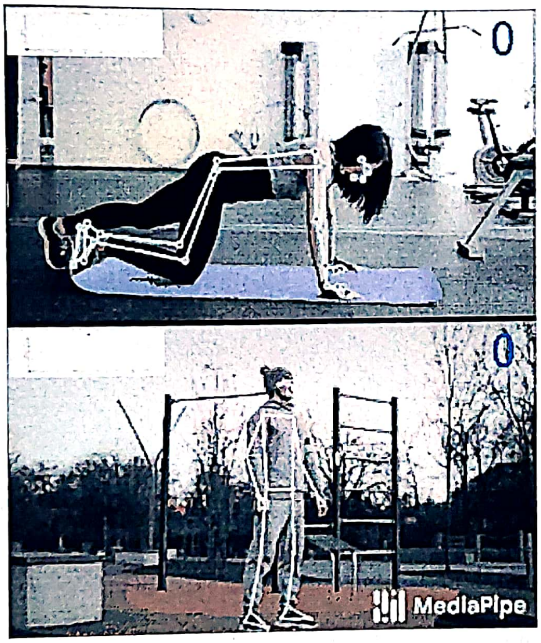
5.2. MediaPipe Solutions

Solutions are open-source pre-built examples based on a specific pre-trained TensorFlow or TFLite model. You can check Solution specific models here. MediaPipe Solutions are built on top of the Framework. Currently, it provides sixteen solutions as listed below.

1. Face Detection
2. Face Mesh
3. Iris
4. Hands
5. Pose
6. Holistic
7. Selfie Segmentation
8. Hair Segmentation
9. Object Detection
10. Box Tracking
11. Instant Motion Tracking
12. Objectron
13. KNIFT
14. AutoFlip
15. MediaSequence
16. YouTube 8M

The solutions are available in C++, Python, JavaScript, Android, iOS, and Coral. As of now, the majority of the solutions are available only in C++ (except KNIFT and IMT) followed by Android, with Python not too far behind.

The other wrapper languages, too, are growing fast with a very active development state. As you can see, even though MediaPipe Framework is cross-platform, that does not imply the same for the solutions. MediaPipe is currently at alpha version 0.7. We can expect the solutions to get more support with the beta releases. Following are some of the solutions provided by MediaPipe.



5.3.BlazePose

A lightweight convolutional neural network architecture for human pose estimation that is tailored for real-time inference on mobile devices. During inference, the network produces 33 body key points for a single person and runs at over 30 frames per second on a Pixel 2 phone. This makes it particularly suited to real-time use cases like fitness tracking and sign language recognition. Our main contributions include a novel body pose tracking solution and a lightweight body pose estimation neural network that uses both heatmaps and regression to keypoint coordinates.

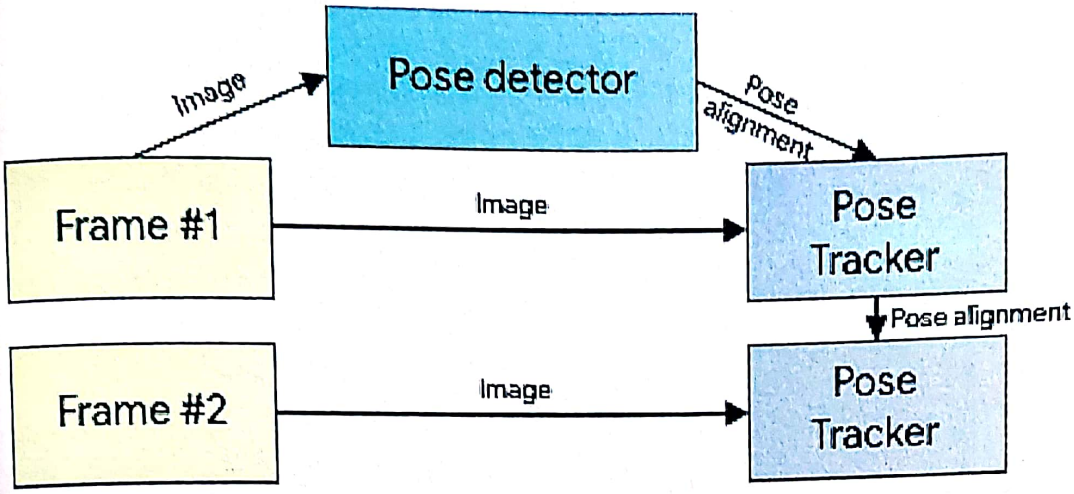
These additional key points provide vital information about face, hands, and feet location with scale and rotation. Together with our face and hand models they can be used to unlock various domain-specific applications like gesture control or sign language without special hardware. With today's release we enable developers to use the same models on the web that are powering MLKit Pose and MediaPipe Python unlocking the same great performance across all devices.

5.3.1.Model deep dive

BlazePose provides real-time human body pose perception in the browser, working up to 4 meters from the camera.
BlazePose trained specifically for highly demanded single-person use cases like yoga, fitness, and dance which require precise tracking of challenging postures, enabling the overlay of digital content and information on top of the physical world in augmented reality, gesture control, and quantifying physical exercises.

For pose estimation, It utilizes proven two-step detector-tracker ML pipeline. Using a detector, this pipeline first locates the pose region-of-interest (ROI) within the frame. The tracker subsequently predicts all 33 pose keypoints from this ROI. Note that for video use cases, the

detector is run only on the first frame. For subsequent frames we derive the ROI from the previous frame's pose keypoints as discussed below.



Credit : https://1.bp.blogspot.com/-WCieLRSZ_MU/YKRhp36pcOI/AAAAAAAAAFM/wjfp2xIzLmkyBYbhuYwLLJanJFZUXoCeCLcBGAsYHQ/s0/Untitled%2B%252831%2529.png

BlazePose's topology contains 33 points extending 17 points by COCO with additional points on palms and feet to provide lacking scale and orientation information for limbs, which is vital for practical applications like fitness, yoga and dance.

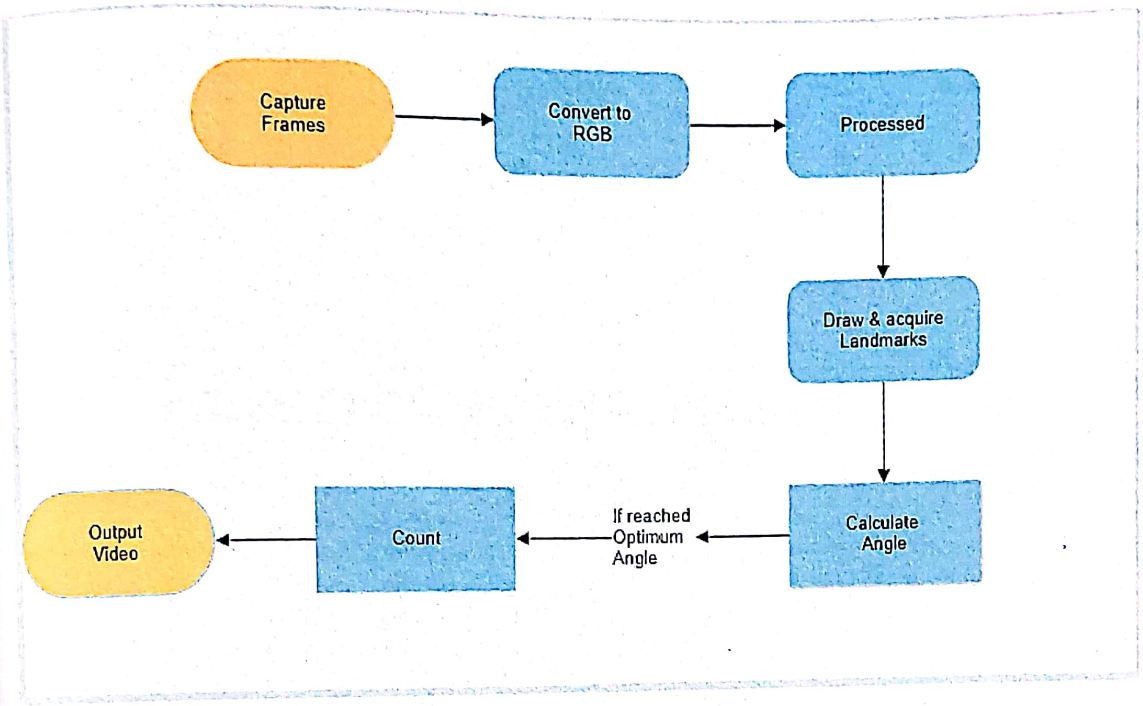
Since the BlazePose CVPR'2020 release, MediaPipe has been constantly improving the models' quality to remain state-of-the-art on the web / edge for single person pose tracking. Besides running through the TensorFlow.js pose-detection API, BlazePose is also available on Android, iOS and Python via MediaPipe and ML Kit. For detailed information, read the Google AI Blog post and the model card.

5.3.2.BlazePose Browser Performance

TensorFlow.js continuously seeks opportunities to bring the latest and fastest runtime for browsers. To achieve the best performance for this BlazePose model, in addition to the TensorFlow.js runtime (w/ WebGL backend) we further integrated with the MediaPipe runtime via the MediaPipe JavaScript Solutions. The MediaPipe runtime leverages WASM to utilize state-of-the-art pipeline acceleration available across platforms, which also powers Google products such as Google Meet.

6. BLOCK DIAGRAM

6. BLOCK DIAGRAM



6.1. Working of Flow Chart:

1. Frames are captured using cv2 library
2. These frames are converted into RGB.
3. Frames are processed.
4. Landmarks are drawn using mediapipe modules.
5. Angle is calculated for designed body position.
6. If the angle is correct it will be added to count.
7. Output video is saved using cv2 library.

7. SETTING UP ENVIRONMENT

7.1.Setting Up Environment

System Configurations :

Operating System : Windows
RAM : 8GB
Storage : 512 GB
Graphic Card : 4GB Nvidia GTX 1650

Python Compiler : Visual Studio Code

Visual Studio Code

Version: 1.69.1 (user setup)
Commit: b06ae3b2d2dbfe28bca3134cc6be65935cdfea6a
Date: 2022-07-12T08:21:24.514Z (1 day ago)
Electron: 18.3.5
Chromium: 100.0.4896.160
Node.js: 16.13.2
V8: 10.0.139.17-electron.0
OS: Windows_NT x64 10.0.22000

VS Code is the best lightweight python compiler and is easy to work with.

Visual Studio Code is a lightweight source code editor. The Visual Studio Code is often called VS Code. The VS Code runs on your desktop. It's available for Windows, macOS, and Linux.

VS Code comes with many features such as IntelliSense, code editing, and extensions that allow you to edit Python source code effectively. The best part is that the VS Code is open-source and free.

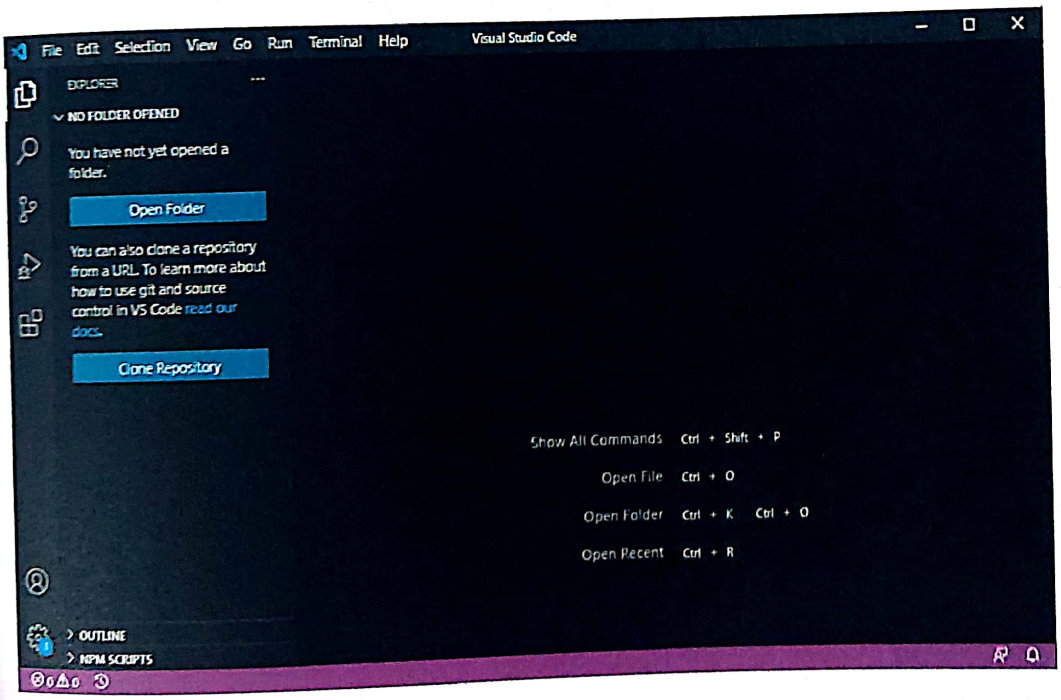
Besides the desktop version, VS Code also has a browser version that you can use directly in your web browser without installing it.

7.1.1.Setting up Visual Studio Code:

To set up the VS Code, you follow these steps:

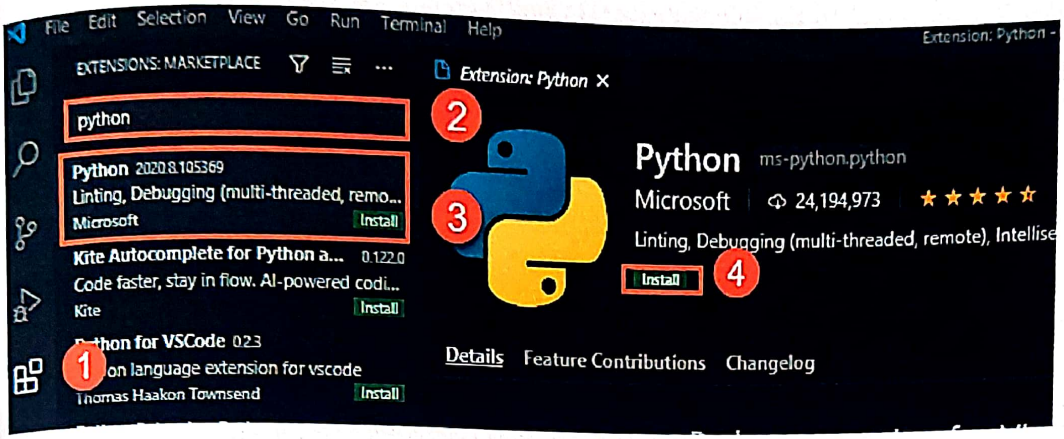
First, navigate to the VS Code official website and download the VS code based on your platform (Windows, macOS, or Linux).

Second, launch the setup wizard and follow the steps.



7.1.2.Installation of Python Extension:

To make the VS Code work with Python, Need to install the Python extension from the Visual Studio Marketplace.



Steps :

- First, click the Extensions tab.
- Second, type the python keyword on the search input.
- Third, click the Python extension. It'll show detailed information on the right pane.
- Finally, click the Install button to install the Python extension.

7.1.3.Creation of Virtual Environment:

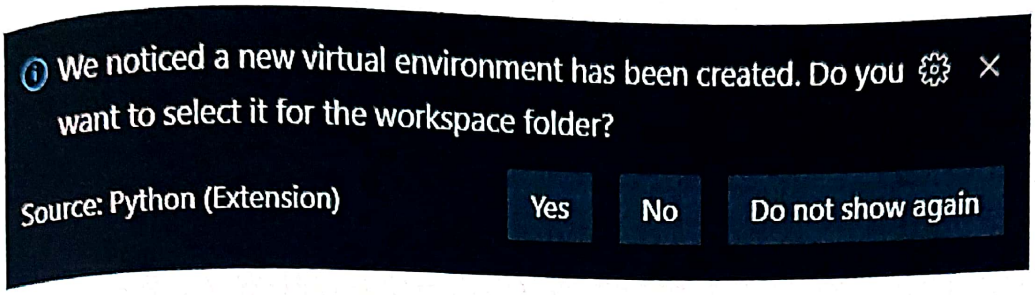
To create a virtual environment, I used the following command, where **"venv"** is the name of the environment folder:

```
# Windows

# You can also use py -3 -m venv venv

python -m venv venv
```

When we create a new virtual environment, a prompt will be displayed to allow you to select it for the workspace.

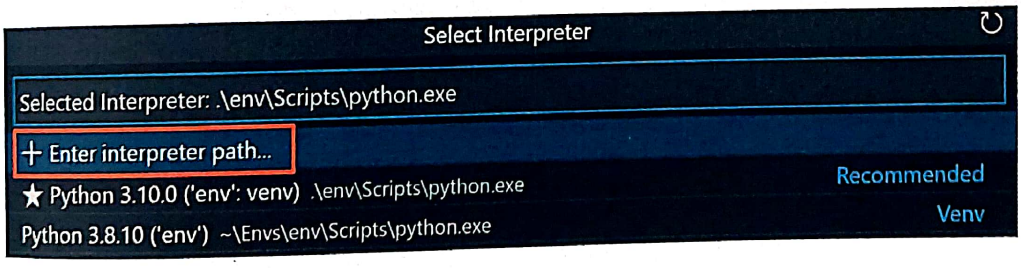


Or

Manually specify an interpreter:

If VS Code doesn't automatically locate an interpreter we want to use, we can browse for the interpreter on your file system or provide the path to it manually.

We can do so by running the Python: Select Interpreter command and clicking on the Enter interpreter path... option that shows on the top of the interpreters list:



We can then either enter the full path of the Python interpreter directly in the text box (for example, ".venv/Scripts/python.exe"), or we can select the Find... button and browse your file system to find the python executable we wish to select.

7.2.Required Libraries

1. Mediapipe

MediaPipe Pose is a ML solution for high-fidelity body pose tracking, inferring 33 3D landmarks and background segmentation masks on the whole body from RGB video frames utilizing our BlazePose research that also powers the ML Kit Pose Detection API. Current state-of-the-art approaches rely primarily on powerful desktop environments for inference, whereas our method achieves real-time performance on most modern mobile phones, desktops/laptops, in python and even on the web.
Mediapipe can be installed using pip. The following command is run in the command prompt to install Mediapipe.

```
pip install mediapipe
```

In Python interpreter, import the package and start using one of the solutions:

```
import mediapipe as mp
```

2. OpenCV

OpenCV is a huge open-source library for computer vision, machine learning, and image processing. OpenCV supports a wide variety of programming languages like Python, C++, Java, etc. It can process images and videos to identify objects, faces, or even the handwriting of a human. When it is integrated with various libraries, such as Numpy which is a highly optimized library for numerical operations, then the number of weapons increases in your Arsenal i.e whatever operations one can do in Numpy can be combined with OpenCV.

OpenCV can be installed using pip. The following command is run in the command prompt to install OpenCV.

```
pip install opencv-python
```

In Python interpreter, import the package and start using one of the solutions:

```
Import cv2
```

3. Numpy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays. It is the fundamental package for scientific computing with Python. It is open-source software. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Windows users can install NumPy via pip command:

pip install numpy

In Python interpreter, import the package and start using one of the solutions:

```
Import numpy as np
```

4. Streamlit

Streamlit is an open-source python framework for building web apps for Machine

Learning and Data Science. We can instantly develop web apps and deploy them easily using Streamlit. Streamlit allows you to write an app the same way you write a python code. Streamlit makes it seamless to work on the interactive loop of coding and viewing results in the web app.

Streamlit can be installed using pip. The following command is run in the command prompt to install Streamlit.

pip install streamlit

In Python interpreter, import the package and start using one of the solutions:

```
Import streamlit as st
```

8. WORKING

8. Working

I wanted to build a web app which is able to count the rep of a workout. For that i used mediapipe and opencv and deployed using streamlit on localhost.

First i imported these libraries

```
import cv2
import mediapipe as mp
import numpy as np
```

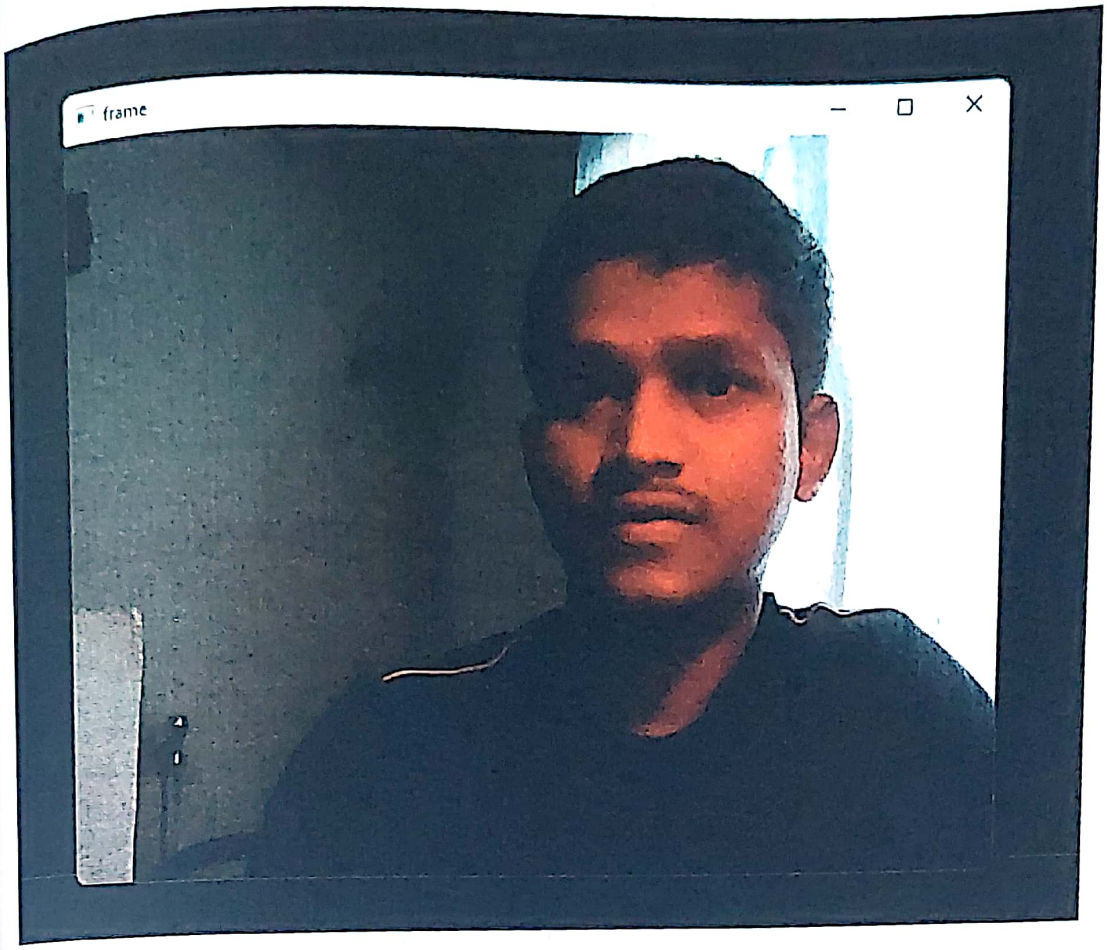
Mediapipe has all the modules which are necessary for detecting all the points on the body. We don't need to draw points on images before feeding video frames.

The module for pose detection is imported and saved in following variables using these commands,

```
mp_drawing = mp.solutions.drawing_utils
mp_pose = mp.solutions.pose
```

As we wanted to capture live video from a webcam, we used the open-cv library to access the camera by using cv2.VideoCapture(0). Parameter 0 specifies using the local cam of the host.

```
cap = cv2.VideoCapture(0)
```



Also created two variables which store the count of reps and position of workout i.e up or down movement.

```
# Curl counter variables
counter = 0
stage = None
```

So for counting a rep we need to reach a certain angle for a particular workout, let consider Squat. A perfect squat has an angle between at least 90 degrees between Hip, Knee and Ankle.

To calculate the angle first we have a function which will be used in a loop while counting the workouts. The function is as follows :

```
def calculate_angle(a,b,c):
    a = np.array(a) # First
    b = np.array(b) # Mid
    c = np.array(c) # End

    radians = np.arctan2(c[1]-b[1], c[0]-b[0]) -
np.arctan2(a[1]-b[1], a[0]-b[0])
    angle = np.abs(radians*180.0/np.pi)

    if angle >180.0:
        angle = 360-angle

    return angle
```

Now let's setup mediapipe instance and get desired output,

```
with mp_pose.Pose(min_detection_confidence=0.5,
min_tracking_confidence=0.5) as pose:
```

In the above line we are calling Pose estimation using mp_pose module with detection and tracking confidence and storing it in pose variable with with function.

Now we read the webcam input and do necessary processing,

```
while cap.isOpened():
    ret, frame = cap.read()
```

We are getting ret and frame from cap.read() function.

The frames received are in BGR format, we will color the images to RGB and make detection and then recolor back to BGR format using cv2 library.

Commands in while loop,

```
# Recolor image to RGB
image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
image.flags.writeable = False

# Make detection
results = pose.process(image)

# Recolor back to BGR
image.flags.writeable = True
image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
```

Now we will draw landmarks on processed images and make detection visible, will be using try and expect to handle errors.

```
# Extract landmarks
try:
    landmarks = results.pose_landmarks.landmark

    # Get coordinates
    hip = [landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].x,landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].y]
    knee = [landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].x,landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].y]
    ankle = [landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].x,landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].y]

    # Calculate angle
    angle = calculate_angle(hip, knee, ankle)

    # Visualize angle
    cv2.putText(image, str(angle),
                tuple(np.multiply(knee, [640, 480]).astype(int)),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255), 2, cv2.LINE_AA
                    )

    # Curl counter logic
    if angle > 160:
        stage = "Up"
    if angle < 110 and stage =='Up':
        stage="Down"
        counter +=1
        print(counter)

except:
    pass
```

In Try , first line we are drawing landmarks and extracting coordinates for hip, knee and ankle in the next three lines and calculating using `calculate_angle` function.

In the fourth line we are visualizing the desired angle using cv2 and in the last 6 we wrote a logic to count the workouts.

We also displayed rep and stage on images of stream video using following commands of cv2:

```
# Render curl counter
# Setup status box
cv2.rectangle(image, (0,0), (225,73), (245,117,16), -1)

# Rep data
cv2.putText(image, 'REPS', (15,12),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,0,0), 1, cv2.LINE_AA)
cv2.putText(image, str(counter),
            (10,60),
            cv2.FONT_HERSHEY_SIMPLEX, 2, (255,255,255), 2, cv2.LINE_AA)

# Stage data
cv2.putText(image, 'STAGE', (65,12),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,0,0), 1, cv2.LINE_AA)
cv2.putText(image, stage,
            (60,60),
            cv2.FONT_HERSHEY_SIMPLEX, 2, (255,255,255), 2, cv2.LINE_AA)


# Render detections
mp_drawing.draw_landmarks(image, results.pose_landmarks, mp_pose.POSE_CONNECTIONS,
                          mp_drawing.DrawingSpec(color=(245,117,66), thickness=2, circle_radius=2),
                          mp_drawing.DrawingSpec(color=(245,66,230), thickness=2, circle_radius=2)
                           )
```

At last `cv2.imshow('Mediapipe Feed', image)` will pop up a display for video.

We can quit stream using

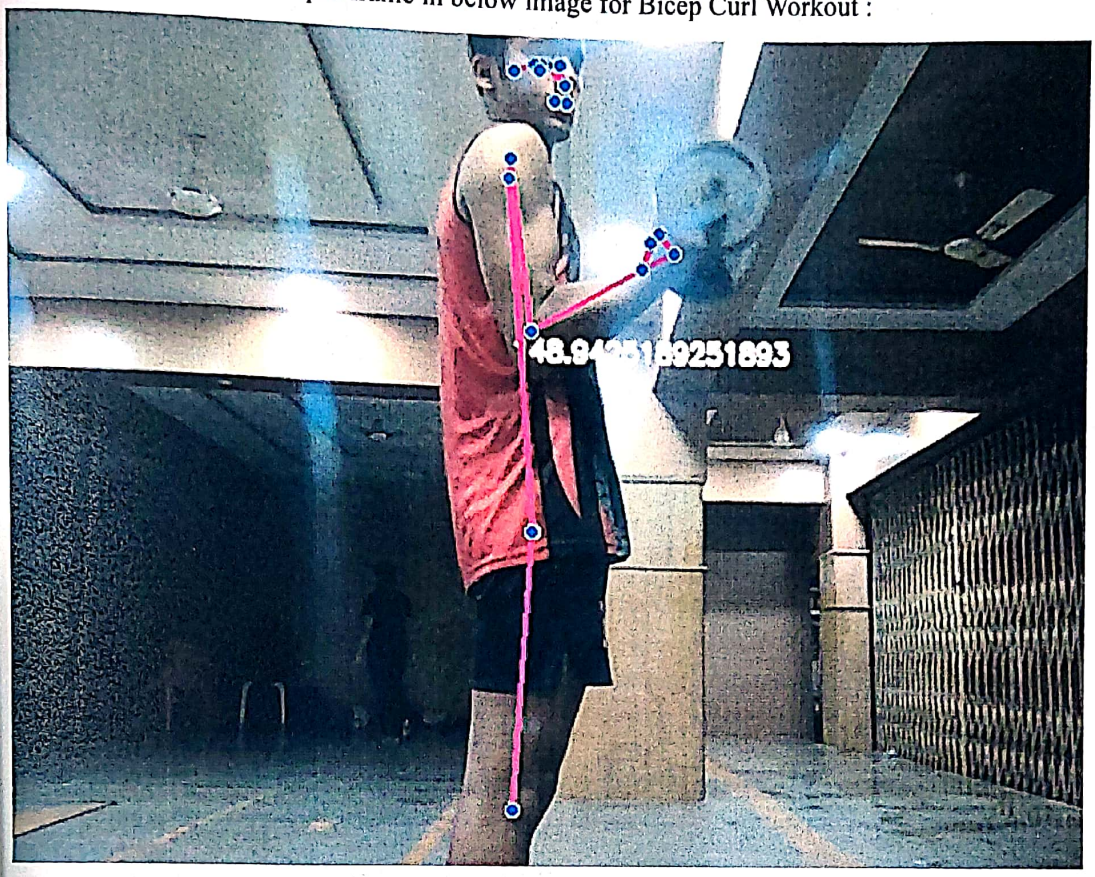
```
if cv2.waitKey(10) & 0xFF == ord('q'):
            break
```

And destroy all the windows for cv2 using following commands,

```
cap.release()
cv2.destroyAllWindows()
```

Output

We can see the desired output frame in below image for Bicep Curl Workout :



Deployment is done on localhost is the help of Streamlit for five different workouts,

9. DEPLOYMENT

9.1.Streamlit

9.1.1. Introduction to Streamlit

Streamlit is an open-source python library for creating and sharing web apps for data science and machine learning projects. The library can help you create and deploy your data science solution in a few minutes with a few lines of code.

Streamlit can seamlessly integrate with other popular python libraries used in Data science such as NumPy, Pandas, Matplotlib, Scikit-learn and many more.

Note: Streamlit uses React as a frontend framework to render the data on the screen.

2. Installation and Set up

Streamlit requires python >= 3.7 version in your machine.

To install streamlit, we need to run the command below in the terminal.

```
pip install streamlit
```

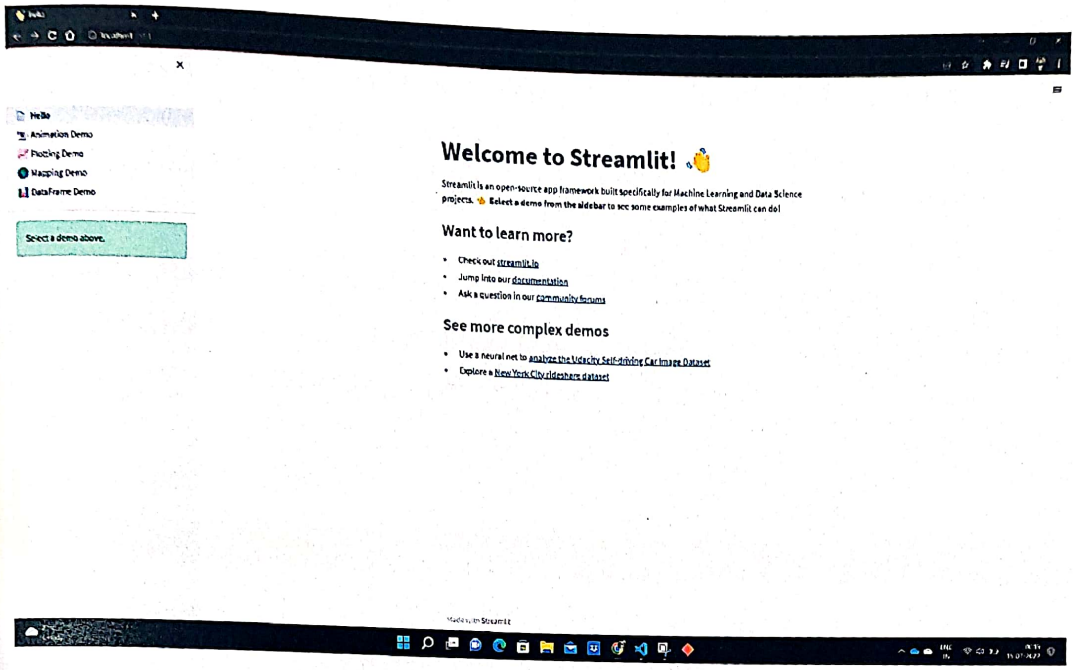
We can also check the version installed on your machine with the following command.

```
streamlit --version
```

After successfully installing streamlit, we can test the library by running the command below in the terminal.

```
streamlit hello
```

Streamlit's Hello app will appear in a new tab in the web browser.



9.2.Deployment

Streamlit is by far the fastest way to create a Web API. By using Streamlit I have deployed a model on localhost. Deployment on servers is a little bit tricky.
Now let's focus on code and output of chunks of code which is used to create this Web API.

9.2.1. Page Title

We can write page title just by using St.title() function.

```
st.title('Workout Counter with MediaPipe')
```

Output :

Workout Counter with MediaPipe

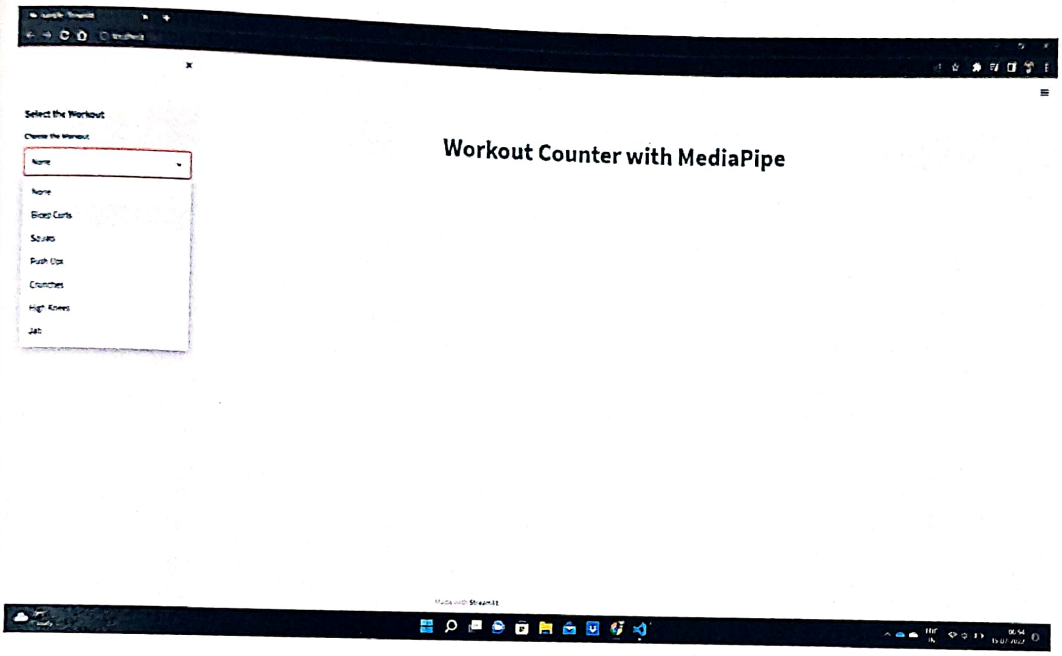
9.2.3. Side bar

1. Dropdown

A Drop down menu for selecting workout is created by using following code:

```
st.sidebar.subheader('Select the Workout')
app_mode = st.sidebar.selectbox('Choose the Workout',
            ["None",'Bicep Curls','Squats','Push Ups','Crunches','High Knees','Jab'])
```

Output :



2. Buttons & Slider

Three button are also created for Starting Video , Stopping Video and Showing of Recorded video by using following codes:

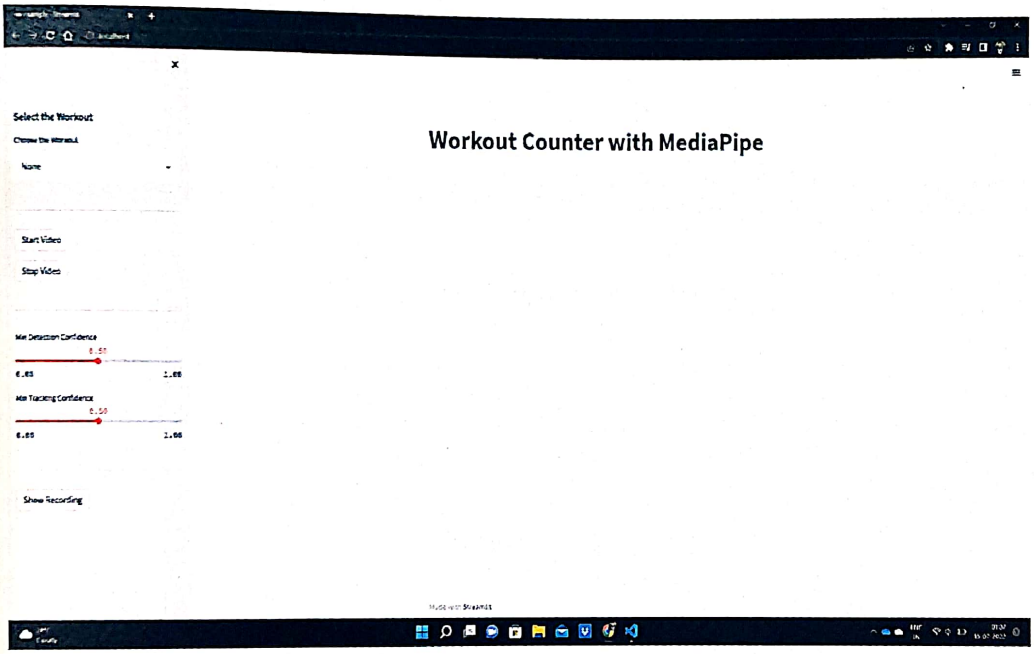
```
st.sidebar.markdown('---')
use_webcam = st.sidebar.button('Start Video')
close_cam = st.sidebar.button('Stop Video')
```

```
outVid = st.sidebar.button('Show Recording')
```

Two sliders also created for `'Min Detection Confidence'` and `'Min Tracking Confidence'` using following codes:

```
detection_confidence = st.sidebar.slider('Min Detection
Confidence', min_value  =0.0,max_value = 1.0,value = 0.5)
tracking_confidence = st.sidebar.slider('Min Tracking
Confidence', min_value = 0.0,max_value = 1.0,value = 0.5)
```

Output :



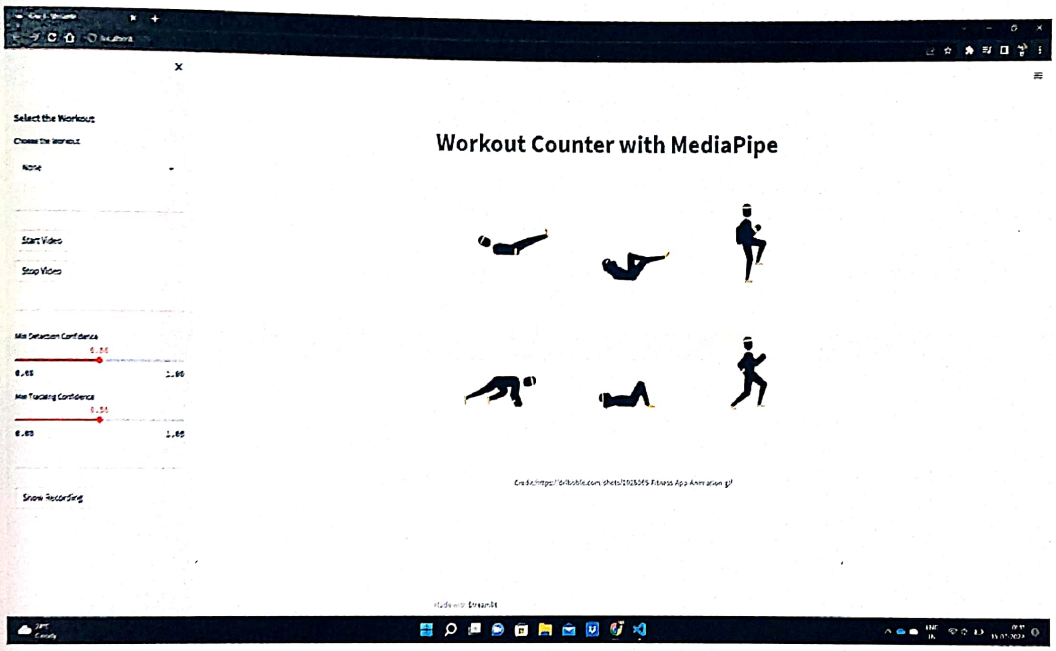
9.3. Workouts

9.3.1. None

When then Web App will show a image of workout,

```
if app_mode == 'None':
    st.image('demo4.gif',
             "Credit:https://dribbble.com/shots/
             2928065-Fitness-App-Animation-gif")
```

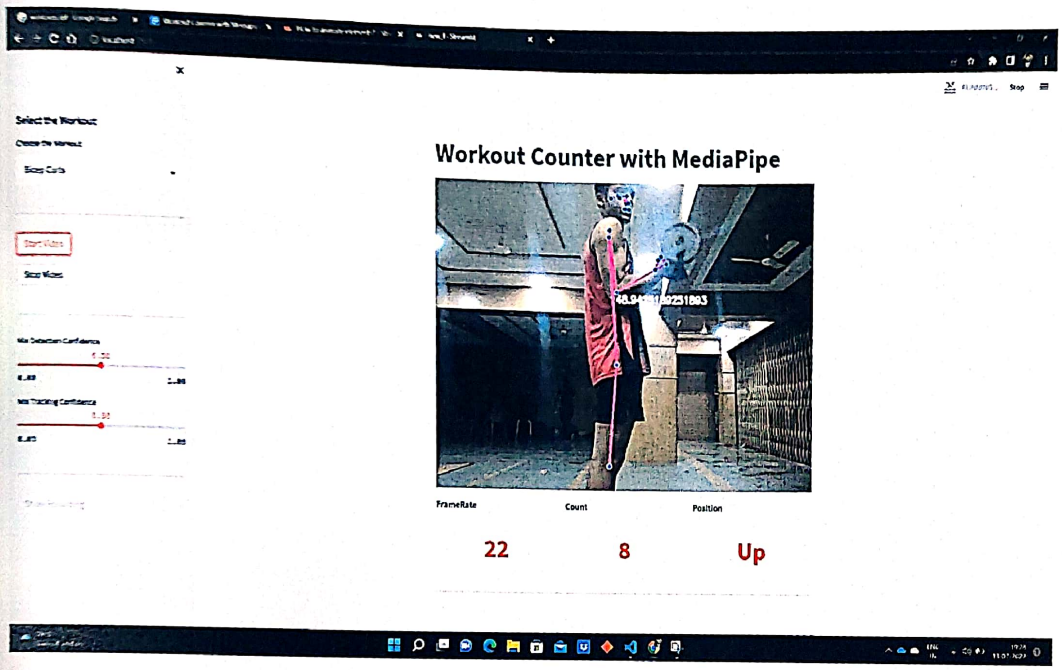
Output:

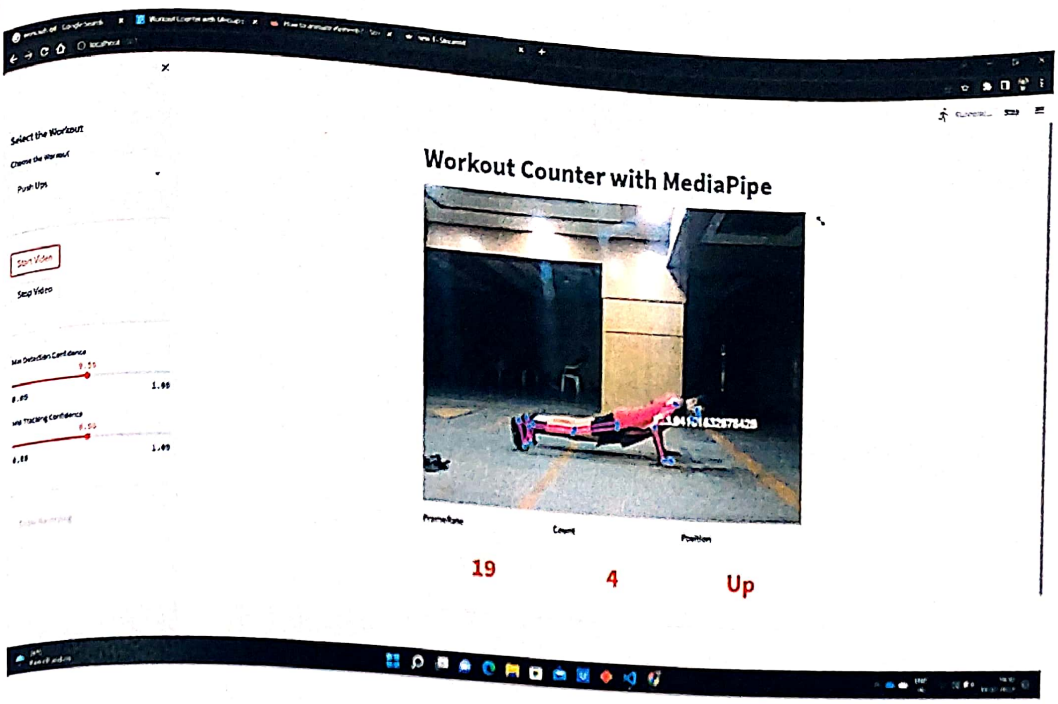


9.3.2. 'Bicep Curls', 'Push Ups',' Jab'

When we select any of above mentioned workout it will execute whole working process amd generate desired results on frames:

Output :

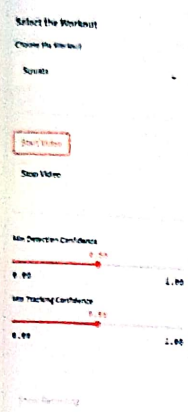




9.3.3. 'Squats' and 'High Knees'

When we select any of above mentioned workout it will execute whole working process and generate desired results on frames:

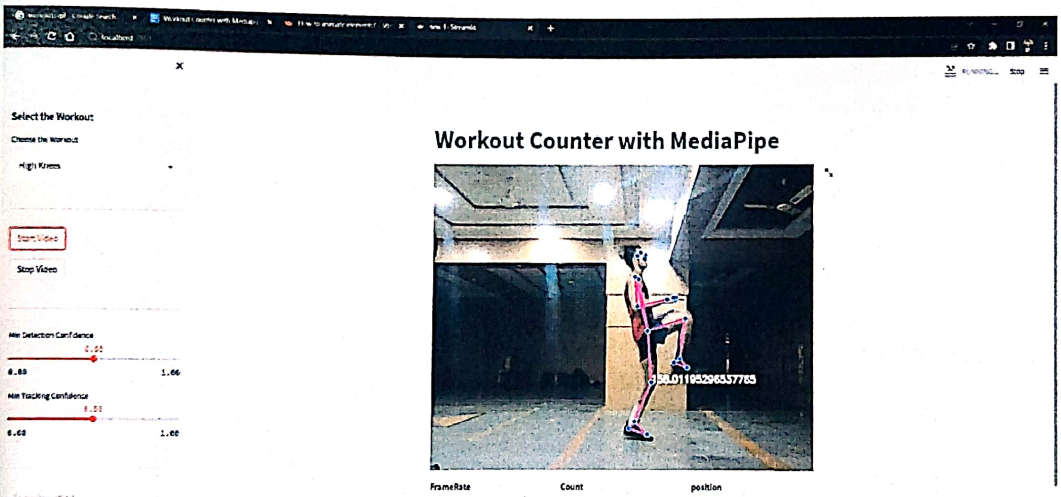
Output:



Workout Counter with MediaPipe

FrameRate Count position

16 3 Down



Workout Counter with MediaPipe

FrameRate Count position

15 1 Down

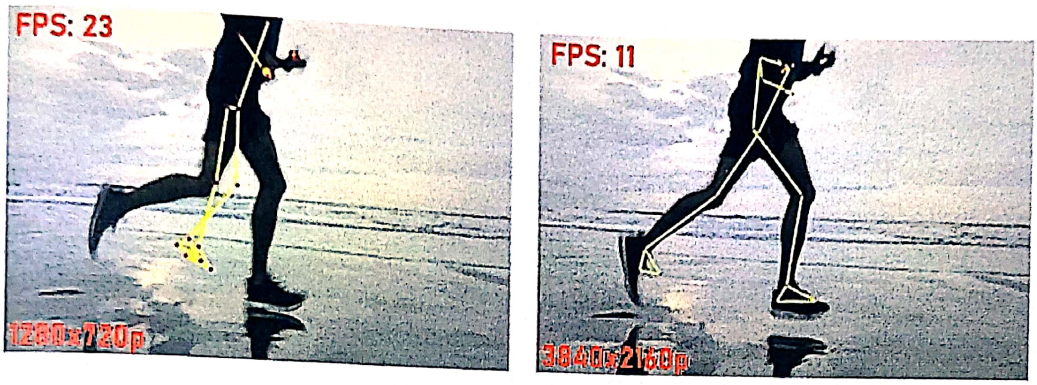
10. LIMITATIONS

10. Limitations

MediaPipe Solutions do not always work in real-time.

The solutions are built on top of MediaPipe Framework that provides Calculator API (C++), Graph construction API (Protobuf), and Graph Execution API (C++, Java, Obj-C). With the APIs, we can build our graph and write custom calculators.

The Toolkit is excellent, but its performance depends on the underlying hardware. The following example shows side by side inference comparison on HD(1280×720) and Ultra HD(3840×2160) video.



You can try building MediaPipe solutions from scratch and certainly see a jump in performance. However, you may still not achieve real-time inferencing.

11. FUTURE SCOPE

11. FUTURE SCOPE

There are various ways this research can be improved in the future. Firstly, the counter is not that accurate and I'm using only one angle of the body to count. We can build a model which can not only count but can also show if the form of workout is correct or not.

We can modify our code which can count not only live video but on recorded video as well and display the right results. We can also include a personal trainer type of module which can give feedback in audio.

12. CONCLUSION

12. CONCLUSION

There are several applications for pose detection in real-life. Here, we delve into one such application to learn more about pose detection. A Smartphone or tablet with a high-resolution camera has the same capabilities as a laptop or desktop. Since mobile phones and tablets have these capabilities, pose estimation has been pushed to new heights. We present an application for monitoring workouts without any involvement of a personal trainer. The application counts the workouts. The system is limited for workout purposes with single-person compatibility at a time.

13. REFERENCES

13. REFERENCES

https://github.com/nicknochnack/MediaPipePoseEstimation

https://www.youtube.com/watch?v=wyWmWaXapmI&ab_channel=AugmentedStartups

https://google.github.io/mediapipe/solutions/pose.html

https://learnopencv.com/introduction-to-mediapipe/

https://learnopencv.com/building-a-body-posture-analysis-system-using-mediapipe/

https://blog.tensorflow.org/2021/05/high-fidelity-pose-tracking-with-mediapipe-blazepose-and-tf
js.html#:~:text=pose%2Ddetection%20API.-,BlazePose%20is%20a%20high%2Dfidelity%20bo
dy%20pose%20model%20designed%20specifically,a%20couple%20of%20years%20ago.

https://www.pythontutorial.net/getting-started/setup-visual-studio-code-for-python/

https://code.visualstudio.com/docs/python/environments

https://arxiv.org/pdf/2006.10204.pdf

https://ijcrt.org/papers/IJCRT2112453.pdf

https://google.github.io/mediapipe/solutions/pose.html

https://www.googlee.com/